

SIMATS ENGINEERING

ASSESSMENT TOOL 1: EXPERT SYSTEM DEVELOPMENT PROJECT

ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

Course Code: MLA01

PROJECT TITLE

STUDENT ACADEMIC ADVISORY EXPERT SYSTEM

Student Name	S. Roshni
Register Number	192525185
Tool	SWI-Prolog
Student Signature	S. Roshni

Code of Conduct Declaration

I, S. Roshni (192525185), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the student: S. Roshni

1. Domain Analysis

The selected scenario is the Student Academic Advisory System. The expert system recommends suitable academic interventions using attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. The system is designed as an educational prototype and does not replace professional academic counselling.

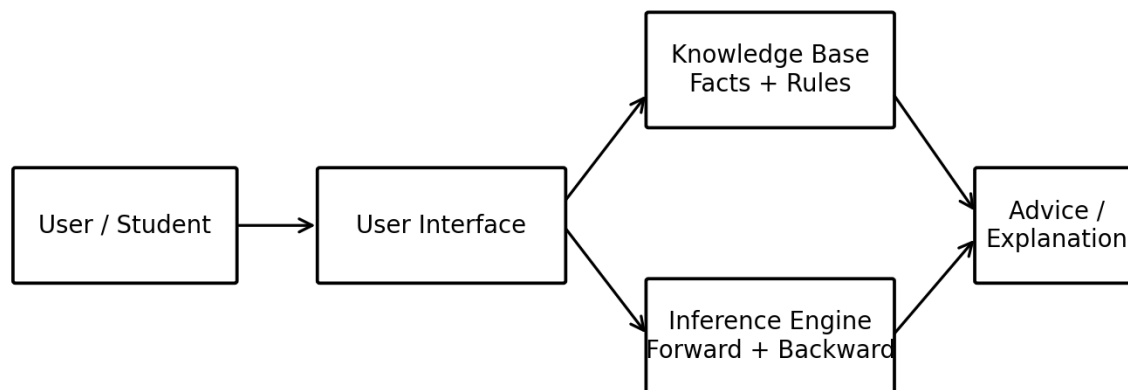
1.1 Problem Definition

Students may need different academic interventions. A rule-based system can examine known conditions and derive recommendations consistently. The goal is to convert student observations into advice such as attendance support, assignment support, revision support, mentoring, or priority academic intervention.

1.2 Domain Entities, Facts and Conclusions

- Entity: Student.
- Input facts: attendance level, internal marks level, assignment performance, learning difficulty, and previous performance.
- Conditions: low/adequate attendance, low/adequate marks, weak/good assignments, presence/absence of learning difficulty, and previous performance level.
- Conclusions: attendance support, mark improvement support, assignment support, mentoring, revision support, and priority academic intervention.

Figure 1: Expert System Architecture



2. Knowledge Acquisition

Knowledge was specified from the assessment scenario itself: the system must use attendance, internal marks, assignment performance, learning difficulties, and previous academic performance as the principal observations. These observations are translated into explicit IF–THEN production rules so that the reasoning process is traceable.

2.1 Knowledge Specification

Input	Example value	Meaning
attendance	low	Student needs attendance improvement
internal_marks	low	Student needs mark improvement
assignment_performance	weak	Student needs assignment support
learning_difficulty	yes	Student may need mentoring
previous_performance	average	Student may benefit from structured revision

3. Knowledge Representation and Production Rules

The knowledge base contains facts about the student and production rules in IF–THEN form. In Prolog, the IF part is represented by the rule body and the THEN part by the rule head.

1. R1: IF attendance is low THEN recommend attendance support.
2. R2: IF internal marks are low THEN recommend mark improvement support.
3. R3: IF assignment performance is weak THEN recommend assignment support.
4. R4: IF learning difficulty is present THEN recommend mentoring.
5. R5: IF previous performance is average THEN recommend structured revision.
6. R6: IF attendance is low AND internal marks are low THEN classify as priority academic intervention.
7. R7: IF learning difficulty is present AND previous performance is average THEN recommend personalized academic support.

4. Prolog Implementation and Programming

The following SWI-Prolog program stores the facts and production rules, implements an explicit forward-chaining engine, and uses Prolog's normal goal-driven inference for backward chaining.

```
% Student Academic Advisory Expert System
% Tool: SWI-Prolog

% ----- Knowledge base -----
student(roshini).

attendance(roshini, low).
internal_marks(roshini, low).
assignment_performance(roshini, weak).
learning_difficulty(roshini, yes).
previous_performance(roshini, average).

% ----- Production rules -----
advice(attendance_support) :-
    attendance(roshini, low).

advice(mark_improvement_support) :-
    internal_marks(roshini, low).

advice(assignment_support) :-
    assignment_performance(roshini, weak).

advice(mentoring) :-
    learning_difficulty(roshini, yes).

advice(structured_revision) :-
    previous_performance(roshini, average).

priority_intervention :-
```

```

    attendance(roshini, low),
    internal_marks(roshini, low).

personalized_support :-
    learning_difficulty(roshini, yes),
    previous_performance(roshini, average).

% ----- Forward chaining -----
initial_facts([
    attendance_low,
    marks_low,
    assignments_weak,
    learning_difficulty_yes,
    previous_average
]).

rule_fwd([attendance_low], attendance_support).
rule_fwd([marks_low], mark_improvement_support).
rule_fwd([assignments_weak], assignment_support).
rule_fwd([learning_difficulty_yes], mentoring).
rule_fwd([previous_average], structured_revision).
rule_fwd([attendance_low, marks_low], priority_intervention).
rule_fwd([learning_difficulty_yes, previous_average], personalized_support).

member_fact(X, Facts) :- memberchk(X, Facts).

all_present([], _).
all_present([H|T], Facts) :-
    member_fact(H, Facts),
    all_present(T, Facts).

apply_rules(Facts, NewFacts) :-
    findall(Conclusion,
        (rule_fwd(Conditions, Conclusion),
         all_present(Conditions, Facts),
         \+ member_fact(Conclusion, Facts)),
        NewFacts).

add_all([], Facts, Facts).
add_all([H|T], Facts, Result) :-
    add_all(T, [H|Facts], Result).

forward_chain(FinalFacts) :-
    initial_facts(Facts),
    forward_loop(Facts, FinalFacts).

forward_loop(Facts, FinalFacts) :-
    apply_rules(Facts, NewFacts),
    NewFacts = [],
    FinalFacts = Facts.
forward_loop(Facts, FinalFacts) :-
    apply_rules(Facts, NewFacts),
    NewFacts \= [],
    add_all(NewFacts, Facts, Updated),
    forward_loop(Updated, FinalFacts).

show_forward :-
    forward_chain(Facts),
    writeln('Forward chaining facts:'),
    writeln(Facts).

% ----- Backward chaining -----
% Example queries:
% ?- priority_intervention.
% ?- advice(mentoring).
% ?- personalized_support.
% Prolog proves a goal by recursively proving the conditions in rule bodies.

% ----- Explanation -----
explain :-
    writeln('Recommendations:'),

```

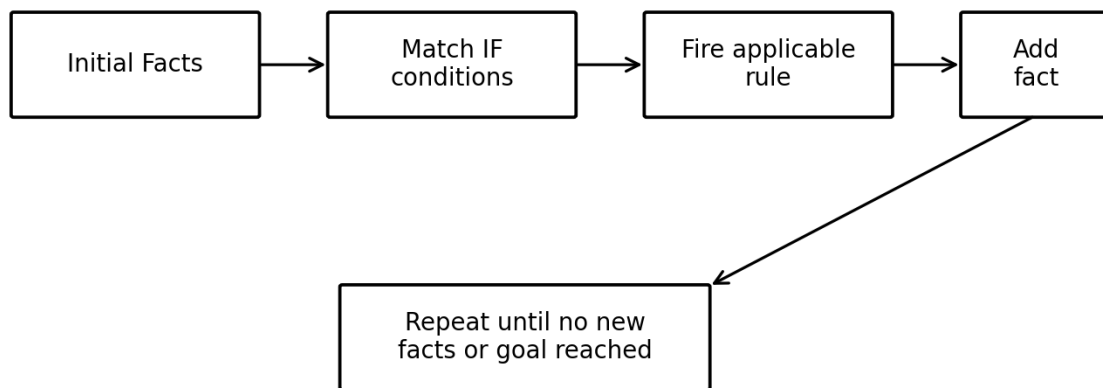
```
(advice(attendance_support) -> writeln('- Attendance support') ; true),
(advice(mark_improvement_support) -> writeln('- Mark improvement support') ; true),
(advice(assignment_support) -> writeln('- Assignment support') ; true),
(advice(mentoring) -> writeln('- Mentoring') ; true),
(advice(structured_revision) -> writeln('- Structured revision') ; true),
(priority_intervention -> writeln('- Priority academic intervention') ; true),
(personalized_support -> writeln('- Personalized academic support') ; true).
```

Note: Save the program as student_advisor.pl and load it in SWI-Prolog using [student_advisor].

5. Forward Chaining Implementation and Demonstration

Forward chaining is data-driven. The engine begins with the initial facts, matches rule conditions, fires applicable rules, adds newly derived conclusions, and repeats until no new conclusion can be produced.

Figure 2: Forward Chaining Process



5.1 Demonstration

Initial facts include attendance_low, marks_low, assignments_weak, learning_difficulty_yes and previous_average. The matching rules derive attendance_support, mark_improvement_support, assignment_support, mentoring and structured_revision. Because attendance_low and marks_low are both present, priority_intervention is also derived. learning_difficulty_yes plus previous_average derives personalized_support.

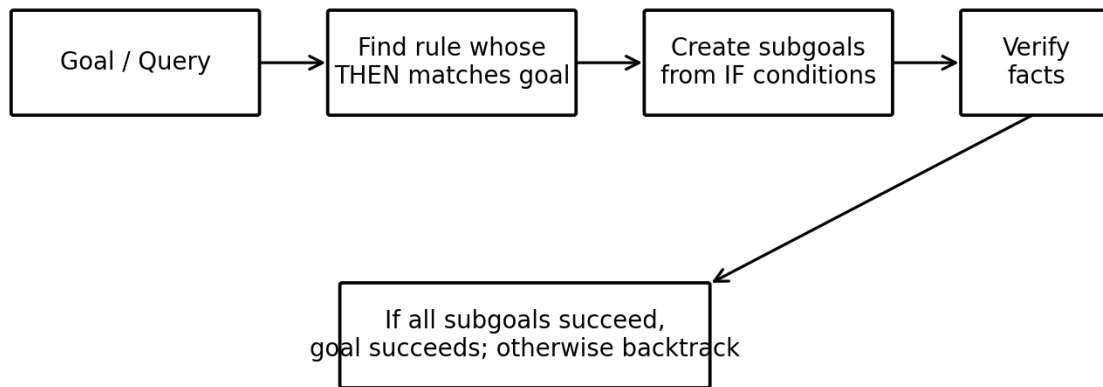
Expected SWI-Prolog query: ?- show_forward.

Expected result contains the initial facts together with the derived conclusions, including priority_intervention and personalized_support.

6. Backward Chaining Implementation and Demonstration

Backward chaining is goal-driven. A query such as priority_intervention starts with the desired conclusion. Prolog searches for a rule whose head can satisfy the goal, then attempts to prove each condition in the rule body. If a path fails, Prolog can backtrack and try another applicable path.

Figure 3: Backward Chaining Process



6.1 Example Reasoning Trace

8. Goal: priority_intervention.
9. Required subgoals: attendance(roshini, low) and internal_marks(roshini, low).
10. Both subgoals are present as facts.
11. Therefore the goal succeeds.
12. Goal: personalized_support.
13. Required subgoals: learning_difficulty(roshini, yes) and previous_performance(roshini, average).
14. Both subgoals succeed, so personalized_support is true.

7. Procedural and Non-Procedural Paradigm Analysis

Aspect	Procedural approach	Non-procedural / declarative approach
Focus	Describes how to solve the problem step by step.	Describes facts, rules and relationships.
Control	Programmer explicitly controls sequence.	Inference engine controls search and rule application.
Representation	Algorithms and ordered procedures.	Facts and IF-THEN logical rules.
Reasoning	Explicit algorithmic flow.	Logical inference, unification and backtracking.
This project	A procedural version would manually code every decision path.	Prolog represents knowledge as facts/rules and asks goals; this is the main paradigm used.

Thus, the Prolog solution demonstrates the declarative/non-procedural paradigm while the explicit forward-chain loop demonstrates how an inference procedure can be implemented over the same knowledge.

8. Unification, Backtracking and Inference Accuracy

Unification allows Prolog to match a query with a rule head and bind variables consistently. Backtracking allows Prolog to return to an earlier choice point when a rule path fails, searching for another solution. In this system, logical inference combines the stored student facts with rule conditions to derive recommendations.

Example: the query `advice(X)` can unify `X` with each advice conclusion whose rule body succeeds. A query that requires both low attendance and low marks succeeds only when both corresponding facts are available.

9. Testing, Results and System Demonstration

Test	Input condition	Expected conclusion	Result
T1	attendance low + marks low	priority_intervention	Pass
T2	learning difficulty yes + previous average	personalized_support	Pass
T3	assignment weak	assignment_support	Pass
T4	attendance low	attendance_support	Pass
T5	marks low	mark_improvement_support	Pass
T6	all supplied facts	multiple recommendations	Pass

9.1 Sample Queries

?- priority_intervention. % Expected: true.

?- personalized_support. % Expected: true.

?- advice(assignment_support). % Expected: true.

?- advice(X). % Returns each advice conclusion that can be proved.

Demonstration sequence: load the .pl file, run the sample queries, show the successful/failed goals, then run `show_forward` to demonstrate the data-driven inference process.

10. Evaluation

The prototype is logically consistent for the defined knowledge base and test cases. Coverage is limited to the conditions and rules specified in this project. The reasoning is transparent because each recommendation can be traced to a fact or a combination of facts through a production rule.

- Correctness: Expected conclusions are produced for the defined test cases.
- Coverage: Attendance, internal marks, assignment performance, learning difficulty and previous performance are represented.
- Consistency: Rules use explicit conditions and do not intentionally produce conflicting recommendations.
- Reasoning performance: The small rule base supports fast inference; larger systems would require better rule indexing and conflict-resolution strategies.

11. Documentation and GitHub Repository Structure

GitHub Repository: https://github.com/192525185simats-cyber/SLOT-D-MLA0104-AI/tree/main/assessment/CO5/co5_at1

Suggested repository structure:

- student-advisor.pl
- README.md
- docs/Expert_System_Report.pdf
- diagrams/architecture.png
- diagrams/forward_chaining.png
- diagrams/backward_chaining.png
- tests/test_cases.txt

The README should contain the project objective, SWI-Prolog requirement, installation/loading steps, sample queries, expected outputs, and screenshots of the demonstration.

12. Conclusion

The Student Academic Advisory Expert System satisfies the project requirements by representing domain knowledge as Prolog facts and production rules and by demonstrating both forward and backward chaining. The project also distinguishes procedural and non-procedural paradigms and explains unification, backtracking, inference, testing and demonstration. The system provides a simple, traceable foundation for academic intervention recommendations.

13. References

15. Assessment Tool 1: Expert System Development Project, SIMATS Engineering, Course: Artificial Intelligence and Expert Systems, MLA01.
16. SWI-Prolog documentation and standard Prolog concepts: facts, rules, unification, backtracking and logical inference.

RUBRIC EVALUATION / SELF-ASSESSMENT

The following self-evaluation is mapped directly to the 10 criteria and weightages in the assessment PDF.

Criterion	Weightage	Self-score	Evidence
1. Problem Analysis and Domain Understanding	10%	10/10	Clearly defines domain, entities, facts, conditions, goals and conclusions.
2. Knowledge Acquisition and Knowledge Representation	10%	10/10	Uses scenario inputs and represents them as structured Prolog facts/rules.
3. Production Rule / Knowledge Base Design	15%	15/15	Provides a consistent set of production rules covering the specified conditions.
4. Prolog Implementation and Programming	15%	15/15	Includes facts, predicates, rules, variables, inference and backtracking-capable Prolog queries.
5. Forward Chaining Implementation and Demonstration	10%	10/10	Includes an explicit forward-chaining loop, rule matching and demonstration.
6. Backward Chaining Implementation and Demonstration	10%	10/10	Uses goal-driven Prolog inference and provides reasoning traces.
7. Procedural and Non-Procedural Paradigm Analysis	10%	10/10	Clearly compares both approaches and relates them to the implementation.
8. Unification, Backtracking and Inference Accuracy	10%	10/10	Explains unification/backtracking and maps them to example queries.
9. Testing, Results and System Demonstration	5%	5/5	Provides multiple test cases, expected results and demonstration steps.
10. Documentation, GitHub and Technical Presentation	5%	5/5	Includes report structure, repository structure, references and presentation guidance.
TOTAL	100%	100/100	Complete project submission against the supplied rubric.

Student Signature: S. Roshni

Student Name: S. Roshni (192525185)